

Systèmes Informatiques

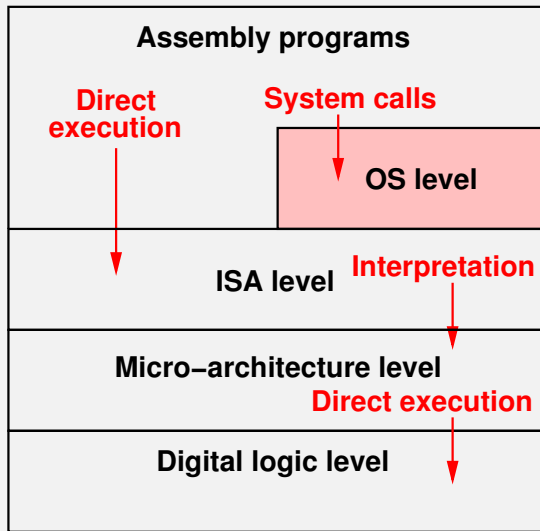
6. Systèmes d'exploitation – gestion de processus

Guillaume Pierre

ISTIC, L3 MIAGE

Table of Contents

- 1** Introduction
- 2 Les appels système
- 3 Les processus
- 4 La communication inter-processus
- 5 Les sémaphores
- 6 Le processus de démarrage d'un OS



Qu'est-ce qu'un système d'exploitation?

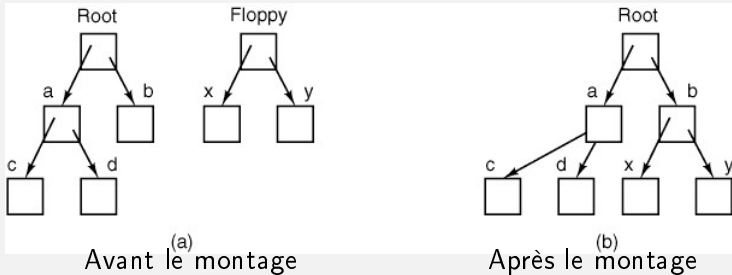
- Un système d'exploitation est une **machine étendue**
 - Il implémente des opérations de plus haut niveau que la couche ISA
 - Ces nouvelles opérations sont fournies sous forme d'**appels système**
 - Le système d'exploitation est une couche hybride: les programmes peuvent aussi utiliser la couche ISA directement
- Un système d'exploitation est un **gestionnaire de ressources**
 - Permettre à plusieurs programmes de s'exécuter en même temps
 - Permettre à plusieurs programmes de partager les ressources matérielles (mesurer l'utilisation des ressources, donner/retirer les droits d'accès aux ressources, arbitrer les conflits, etc.)

- Un processus est un **programme en train de s'exécuter**
 - Espace mémoire privé (isolé des autres processus)
 - Pile privée, registres PC et SP, etc.
 - Un processus est toujours créé par un autre processus
- L'OS prend en charge l'**ordonnancement des processus**
 - A un instant donné il ne peut s'exécuter qu'un seul programme sur un CPU donné
 - L'OS fait en sorte que chaque processus reçoive une part équitable des ressources de calcul
- Les processus sont **isolés** les uns des autres
 - Chaque processus a l'illusion de pouvoir utiliser l'ensemble de la machine
 - La communication entre processus doit avoir lieu à travers des mécanismes explicites de **communication inter-processus**

- Les processus veulent pouvoir **échanger des données** avec des périphériques
- Mais chaque périphérique est différent!
 - Différents types (disques, claviers, réseaux, etc.)
 - Différentes vitesses
 - Différents constructeurs
 - Etc.
- L'OS doit fournir: une **interface homogène** pour accéder à toutes les sortes de périphériques
 - Et également **arbitrer entre plusieurs processus** qui voudraient utiliser le même périphérique au même moment

- Les processus ont besoin de mémoire pour fonctionner
 - Mais on veut isoler l'espace mémoire des différents processus
 - Mieux: donner à chaque processus l'illusion qu'**il a accès à l'intégralité de la mémoire de la machine** sans être perturbé par les autres processus
 - Encore mieux: donner à chaque processus l'illusion qu'**il a accès à l'ensemble de l'espace d'adressage mémoire** (adresses $0 - 2^{32}$) indépendamment de la mémoire effectivement présente dans la machine
- L'OS doit fournir:
 - **Isolation mémoire**
 - **Pagination** (c'est-à-dire être capable de déplacer des morceaux de la mémoire vers le disque et vice-versa)
 - **Memory mapping** (c'est-à-dire donner à chaque processus accès à l'ensemble de l'espace d'adressage mémoire)

- Est-ce que quelqu'un ne sait pas ce qu'est un fichier? :-)
 - Comment créer des **fichiers, répertoires, permissions, etc.**
 - En utilisant des blocs de données sur disque
- On peut **monter** plusieurs systèmes de fichiers en une hiérarchie unique



■ Fichiers spéciaux

- Beaucoup de concepts OS sont représentés comme des fichiers
- Périphériques, mécanismes de communication inter-processus, etc.

Table of Contents

- 1 Introduction
- 2 Les appels système**
- 3 Les processus
- 4 La communication inter-processus
- 5 Les sémaphores
- 6 Le processus de démarrage d'un OS

- Les appels système permettent aux processus de **demander une opération** au système d'exploitation

- Appels système pour la **gestion de processus**:

<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, opts)</code>	Wait for a child to terminate
<code>s = wait(&status)</code>	Old version of waitpid
<code>s = execve(name, argv, envp)</code>	Replace a process core image
<code>exit(status)</code>	Terminate process execution and return status
<code>size = brk(addr)</code>	Set the size of the data segment
<code>pid = getpid()</code>	Return the caller's process id
<code>pid = getpgid()</code>	Return the id of the caller's process group
<code>pid = setsid()</code>	Create a new session and return its process group id
<code>l = ptrace(req, pid, addr, data)</code>	Used for debugging

■ Appels système pour la **gestion de signaux**:

<code>s = sigaction(sig, &act, &oldact)</code>	Define action to take on signals
<code>s = sigreturn(&context)</code>	Return from a signal
<code>s = sigprocmask(how, &set, &old)</code>	Examine or change the signal mask
<code>s = sigpending(set)</code>	Get the set of blocked signals
<code>s = sigsuspend(sigmask)</code>	Replace the signal mask and suspend the process
<code>s = kill(pid, sig)</code>	Send a signal to a process
<code>residual = alarm(seconds)</code>	Set the alarm clock
<code>s = pause()</code>	Suspend the caller until the next signal

■ Appels système pour la **gestion de fichiers**:

<code>fd = creat(name, mode)</code>	Obsolete way to create a new file
<code>fd = mknod(name, mode, addr)</code>	Create a regular, special, or directory i-node
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>pos = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &buf)</code>	Get a file's status information
<code>s = fstat(fd, &buf)</code>	Get a file's status information
<code>fd = dup(fd)</code>	Allocate a new file descriptor for an open file
<code>s = pipe(&fd[0])</code>	Create a pipe
<code>s = ioctl(fd, request, argp)</code>	Perform special operations on a file
<code>s = access(name, amode)</code>	Check a file's accessibility
<code>s = rename(old, new)</code>	Give a file a new name
<code>s = fcntl(fd, cmd, ...)</code>	File locking and other operations

■ Appels système pour la **gestion de répertoires et de systèmes de fichiers**:

<code>s = mkdir(name, mode)</code>	Create a new directory
<code>s = rmdir(name)</code>	Remove an empty directory
<code>s = link(name1, name2)</code>	Create a new entry, name2, pointing to name1
<code>s = unlink(name)</code>	Remove a directory entry
<code>s = mount(special, name, flag)</code>	Mount a file system
<code>s = umount(special)</code>	Unmount a file system
<code>s = sync()</code>	Flush all cached blocks to the disk
<code>s = chdir(dirname)</code>	Change the working directory
<code>s = chroot(dirname)</code>	Change the root directory

■ Appels système pour la **gestion des protections**:

<code>s = chmod(name, mode)</code>	Change a file's protection bits
<code>uid = getuid()</code>	Get the caller's uid
<code>gid = getgid()</code>	Get the caller's gid
<code>s = setuid(uid)</code>	Set the caller's uid
<code>s = setgid(gid)</code>	Set the caller's gid
<code>s = chown(name, owner, group)</code>	Change a file's owner and group
<code>oldmask = umask(complmode)</code>	Change the mode mask

■ Appels système pour la **gestion du temps**:

`seconds = time(&seconds)`

Get the elapsed time since Jan. 1, 1970

`s = stime(tp)`

Set the elapsed time since Jan. 1, 1970

`s = utime(file, timep)`

Set a file's "last access" time

`s = times(buffer)`

Get the user and system times used so far

Les appels système sont très similaires à des interruptions

Device:
Send electrical signal to interrupt controller.



Controller:
1. Interrupt CPU.
2. Send digital identification of interrupting device.



Kernel:
1. Save registers.
2. Send notification message to driver
3. Restart a process (probably the driver)

(a)

Interruption matérielle

Caller:
1. Put message pointer and destination of message into CPU registers.
2. Execute software interrupt instruction.



Kernel:
1. Save registers.
2. Send and/or receive message.
3. Restart a process (not necessarily the calling process).

(b)

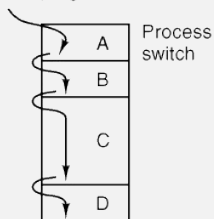
Appel système (logiciel)

Table of Contents

- 1 Introduction
- 2 Les appels système
- 3 Les processus**
- 4 La communication inter-processus
- 5 Les sémaphores
- 6 Le processus de démarrage d'un OS

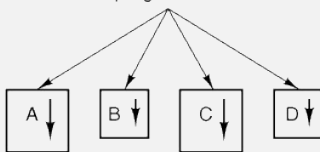
- Comment exécute-t-on **plusieurs processus** simultanément sur un **seul CPU**?
 - (a) Multiprogrammation: alterner entre plusieurs programmes
 - (b) Conceptuellement chaque programme a son propre “program counter” et s'exécute en isolation
 - (c) Mais un seul programme est actif à chaque instant

One program counter

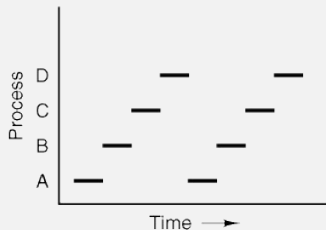


(a)

Four program counters



(b)



(c)

■ Un processus est créé:

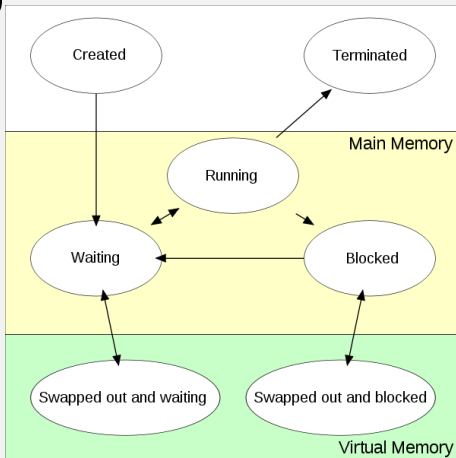
- Lors de l'initialisation du système d'exploitation
- Quand un autre processus lance l'appel système de création de processus

■ Un processus s'arrête:

- Lorsque le programme se termine normalement (arrêt volontaire)
- Lorsque le programme s'arrête pour cause d'erreur (arrêt volontaire)
- Lors d'une erreur fatale (arrêt involontaire)

États d'un processus

- Un processus peut se trouver dans **3 états possibles**:
 - **Running** (il est en train de s'exécuter sur le CPU)
 - **Waiting** (il est prêt à s'exécuter mais il doit attendre son tour)
 - **Blocked** (il ne peut pas s'exécuter car il attend un événement extérieur)

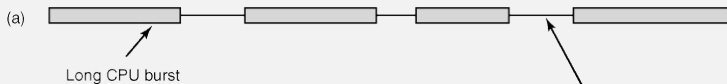


- Concrètement, **un processus est composé de:**
 - Une **image**: le code exécutable du programme qu'il exécute
 - De la **mémoire**: utile pour conserver le programme, les variables, la pile etc.
 - Les **descripteurs des ressources système** qu'il utilise en ce moment (fichiers, etc.)
 - Des **attributs de sécurité**: identifiant du propriétaire du processus, liste des autorisations, etc.
 - **L'état du processus**:
 - Contenu des registres
 - Program counter
 - Stack pointer
 - Signaux en attente
 - Et quelques autres...

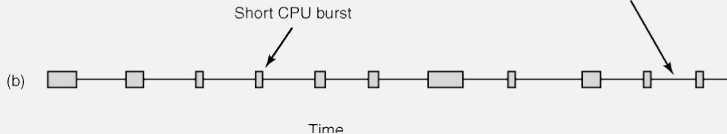
Les détails varient d'un système à un autre...

L'exécution d'un processus est limité par le CPU ou les I/O

**CPU
bound:**



I/O bound:



Ordonnancement des processus

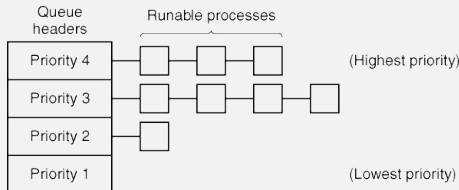
- Quand plusieurs processus sont prêts à s'exécuter, l'**ordonnanceur de processus** doit choisir lequel va effectivement s'exécuter
- Il existe de nombreuses **disciplines d'ordonnancement**:

Round-Robin Chaque processus a la possibilité de s'exécuter pendant une **durée déterminée**. Si un processus n'arrive pas à utiliser son intervalle de temps, on passe au suivant.

- Quelle durée utiliser? Compromis fluidité/efficacité

Priorités On donne à chaque processus un **niveau de priorité**. Le processus en attente qui a la plus haute priorité est toujours choisi.

- Les processus basse priorité peuvent ne jamais s'exécuter!



Fair-share Chaque processus est associé à un **utilisateur**, on essaye de donner une fraction égale des ressources CPU à chaque utilisateur quelque soit le nombre de processus qu'il crée.

Real-time On donne des **garanties fortes** à certains processus pour qu'ils disposent des ressources **quand ils en ont besoin** (périodique / non-périodique).

- Le système peut accepter jusqu'à m événements périodiques de durée C_i secondes à une période P_i si:

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

- Très utilisé pour les systèmes multimédia, les systèmes de contrôle industriel, etc.

■ Changement de contexte coopératif:

- Un processus rend volontairement la main au système d'exploitation
- Cela se produit chaque fois qu'un processus fait un **appel système**

■ Changement de contexte préemptif:

- Le système reprend la main lui-même
- Cela se produit quand un processus a épuisé son **quota de temps**
- Interruption matérielle générée par un timer (initialisé par le système avant d'ordonnancer le processus)

- Un changement de contexte est généralement initié par une **interruption matérielle**

1. Hardware stacks program counter, etc.
2. Hardware loads new program counter from interrupt vector.
3. Assembly language procedure saves registers.
4. Assembly language procedure sets up new stack.
5. C interrupt service runs (typically reads and buffers input).
6. Scheduler marks waiting task as ready.
7. Scheduler decides which process is to run next.
8. C procedure returns to the assembly code.
9. Assembly language procedure starts up new current process.

Table of Contents

- 1 Introduction
- 2 Les appels système
- 3 Les processus
- 4 La communication inter-processus**
- 5 Les sémaphores
- 6 Le processus de démarrage d'un OS

- Les processus sont **isolés** les uns des autres
 - Espace mémoire privé
 - Registre PC privé, pile privée, etc.
 - Chaque processus a l'illusion d'être le seul sur le système

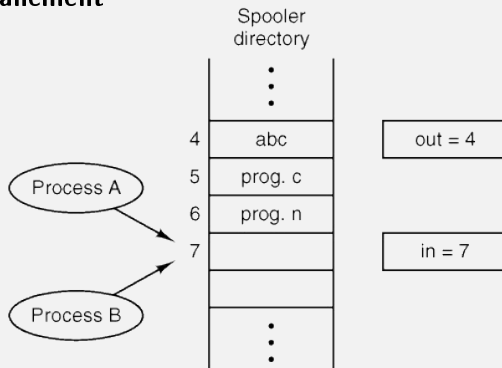
MAIS...

- Les processus doit également pouvoir **communiquer** entre eux
 - Pour synchroniser leurs actions dans le temps
 - Pour échanger des données

- Fichiers
- Signaux
- Pipes
 - Communication FIFO entre deux processus
- Passage de messages
 - `send(destination, message);`
 - `message = receive(source);`
- Mémoire partagée
 - Plusieurs processus accèdent simultanément au **même espace mémoire**
 - Efficace mais dangereux: race conditions!

Les “race conditions”

- Les “race conditions” apparaissent quand **plusieurs processus partagent une ressource en lecture+écriture**
- Exemple: un spooler d'imprimante
 - Chaque fichier à imprimer est enregistré dans un tableau
 - Le processus d'impression lit les fichiers du tableau et les imprime
 - Imaginons que **deux processus ajoutent un fichier dans le tableau simultanément**

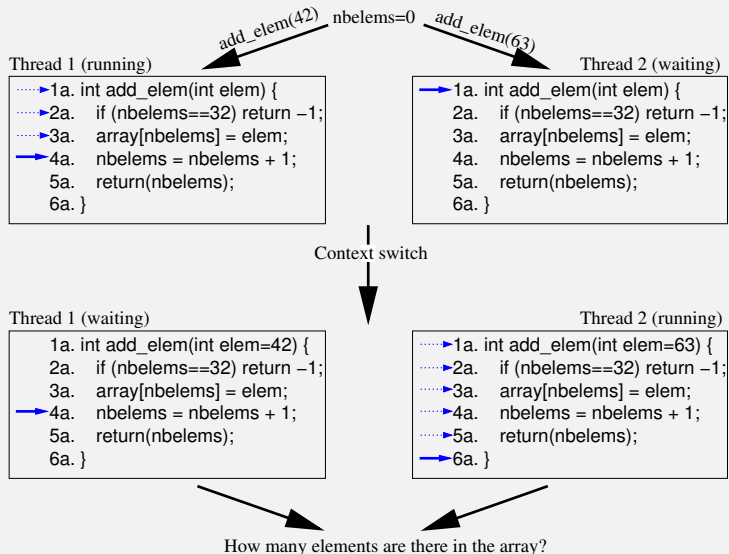


Exemple de “race condition” [1/2]

```
int array[32], nbelems; /* variables allouées en mémoire partagée */

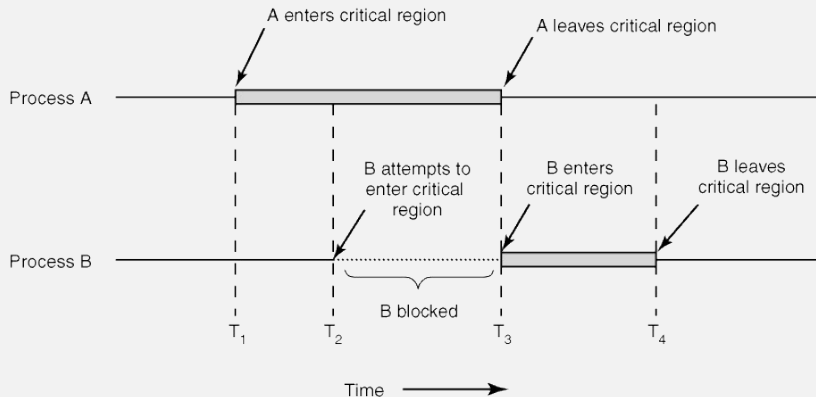
int add_elem(int elem) {
    if (nbelems==32) return -1;
    array[nbelems] = elem;
    nbelems = nbelems + 1;
    return(nbelems);
}
```

Exemple de "race condition" [2/2]



- Nous voulons **interdire à deux processus d'être simultanément dans la même section critique**

- 1 Demander l'autorisation avant d'entrer dans la section critique
- 2 Indiquer quand on en sort



Une **MAUVAISE** implémentation de l'exclusion mutuelle

```
while (TRUE) {  
    while (turn != 0)      /* loop */ ;  
    critical_region( );  
    turn = 1;  
    noncritical_region( );  
}
```

(a)

```
while (TRUE) {  
    while (turn != 1)      /* loop */ ;  
    critical_region( );  
    turn = 0;  
    noncritical_region( );  
}
```

(b)

- Cet algorithme implémente une **alternance stricte**: le processus A ne peut pas entrer en section critique deux fois de suite
- **Attente active**: nous gaspillons des ressources de calcul!

- Tous les CPUs supportent une instruction **Test and Set Lock**:
TSL RX, LOCK
 - Copie le contenu de LOCK dans le register RX
 - Ecrit 1 dans le registre LOCK
 - **Ces deux actions sont atomiques**: aucun processus ne peut accéder à LOCK entre les deux opérations

enter_region:

tsl register, lock	copy lock to register and set lock to 1
cmp register, #0	was lock zero?
jne enter_region	if it was non zero, lock was set, so loop
ret	return to caller; critical region entered

leave_region:

move lock, #0	store a 0 in lock
ret	return to caller

- **Positif**: nous avons résolu le problème de l'alternance stricte
- **Négatif**: nous avons toujours l'attente active

La bonne solution: les mutex

mutex_lock:

TSL REGISTER,MUTEX	copy mutex to register and set mutex to 1
CMP REGISTER,#0	was mutex zero?
JZE ok	if it was zero, mutex was unlocked, so return
CALL thread_yield	mutex is busy; schedule another thread
JMP mutex_lock	try again later

ok: RET | return to caller; critical region entered

mutex_unlock:

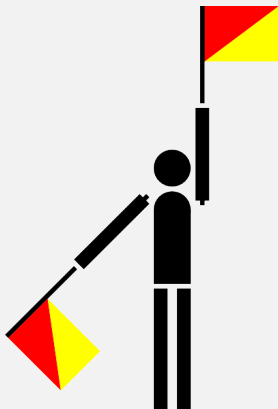
MOVE MUTEX,#0	store a 0 in mutex
RET	return to caller

- **Plus d'attente active!** Le processus vérifie le verrou **une fois** chaque fois qu'il est réveillé
- Version étendue des mutex: **les sémaphores**

Table of Contents

- 1 Introduction
- 2 Les appels système
- 3 Les processus
- 4 La communication inter-processus
- 5 Les sémaphores**
- 6 Le processus de démarrage d'un OS

- Les sémaphores permettent de **synchroniser plusieurs processus**
 - Quand ils doivent partager des ressources (mémoire partagée, etc.)



- Un sémaphore est un **entier naturel** qui possède deux méthodes **UP()** et **DOWN()**:
 - **DOWN()** (parfois appelée **P()**):
 - If $sem \geq 1$ then $sem = sem - 1$
 - Sinon, bloquer le processus
 - **UP()** (parfois appelée **V()**):
 - S'il y a des processus bloqués sur ce sémaphore, alors en débloquent un
 - Sinon, $sem = sem + 1$
- Les opérations sur les sémaphores sont atomiques

- On veut définir des **sections critiques** de code où un seul processus peut entrer à la fois
- Solution: les sémaphores!
 - Il faut initialiser `sem=1`
 - Pour **entrer** dans la section critique: `DOWN(sem)`
 - Pour **sortir** de la section critique: `UP(sem)`
- Comment ajuster le code de `add_elem()` en conséquence?

Exemple

```
int array32, nbelems; /* variables allouées en mémoire partagée */

int add_elem(int elem) {
    int tmp;

    DOWN();
    if (nbelems==32) {
        UP();
        return -1;
    }
    arraynbelems = elem;
    nbelems = nbelems + 1;
    tmp = nbelems;           /* Pourquoi a-t-on besoin de cette variable tmp? */
    UP();
    return(tmp);
}
```

Table of Contents

- 1 Introduction
- 2 Les appels système
- 3 Les processus
- 4 La communication inter-processus
- 5 Les sémaphores
- 6 Le processus de démarrage d'un OS**

Etape 1: le BIOS

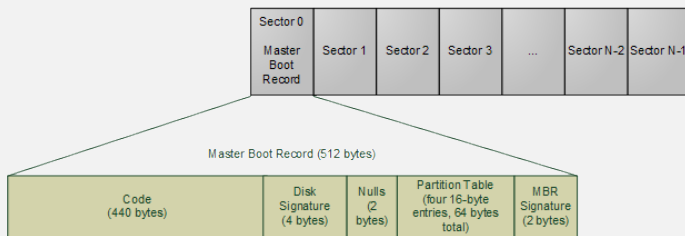
■ BIOS = **B**asic **I**nput/**O**utput **S**ystem

- Ecrit en mémoire non-volatile (mémoire flash)
- Le BIOS s'exécute toujours en premier quand on démarre un ordinateur

■ Séquence d'actions:

- 1 Teste le matériel, vérifie que tout fonctionne bien
- 2 Identifie et initialise les périphériques
- 3 Détermine quels périphériques sont "bootables"
- 4 Choisit l'un des périphériques "bootables", charge le premier secteur en mémoire, exécute le programme (1 secteur = 512 octets)

N-sector disk drive. Each sector has 512 bytes.



Etape 2: le “boot loader”

- Le programme sur le 1er secteur s'appelle un **boot loader**: GRUB, LILO, NTLDR, etc.
 - Objectif du boot loader: trouver/choisir un système d'exploitation, le démarrer
- Séquence d'actions:
 - 1 Charger d'autres sector(s) du périphérique bootable pour charge davantage de code/données/configurations
 - 2 Proposer une liste de choix à l'utilisateur
 - 3 Lire la table de partition du périphérique, charger un fichier en mémoire (p.ex: /boot/vmlinuz-2.6.31-20-generic)
 - Cela demande que le boot loader comprenne la structure de la table de partition et du système de fichiers (au moins en lecture seule)
 - 4 Exécute le programme (noyau)

Etape 3: démarrage du noyau

- Le noyau initialise les interruptions, met le CPU en mode protégé, etc.
- Le noyau est considéré comme le processus 0. Il crée un processus 1 de toute pièce, puis il appelle `cpu_idle()` pour se mettre en veille
- Le process 1 exécute le programme `/sbin/init`: il lit les fichiers de configuration du système d'exploitation, et il démarre les autres processus

